

AI技术应用验证报告

一、报告说明

本文档对应作业步骤三“AI技术应用验证报告”。根据前两步文档的分析，本项目已将AI辅助需求分析、AI驱动代码生成与审查、AI辅助测试用例与质量分析确定为优先改进场景。考虑到本项目“词典编撰辅助平台”具有Graph-RAG检索、多智能体推理、专家闭环验证等特点，测试阶段既要验证普通软件功能，也要验证AI输出是否具备证据链、可解释性和可复核性。中国信通院AI4SE报告指出，智能测试已成为AI赋能软件开发的重要落地方向；智能测试能力标准也强调测试资产生成、测试执行和质量评估的体系建设[1][2]。因此，本次原型验证选择“AI辅助测试用例生成”作为最小可行验证场景。

本次验证的目标不是证明AI可以完全替代测试人员，而是验证AI能否在较短时间内，根据需求描述和过程文档生成较完整的测试用例草案，并通过人工筛选形成可执行的测试资产。验证工具使用OpenAI Codex（GPT-5，Codex桌面环境）这一类大语言模型工具，验证日期为2026年5月30日，验证材料来自本作业已完成的《AI技术调研与场景分析》和《软件过程改进方案设计文档》。

二、验证场景与工具选型

1. 验证场景

验证场景选择为：**针对词典编撰辅助平台的核心功能链路，使用AI生成测试用例**。核心功能链路包括：

- 用户输入字头并发起检索；
- 系统返回义项、书证、来源和置信度；
- Graph-RAG根据字头和义项检索证据材料；
- 保守派Agent、革新派Agent、裁判Agent分别输出观点；
- 专家查看证据链和Agent结论，并提交反馈；
- 系统记录修改日志和专家采纳结果。

这一场景与步骤二中的“AI辅助测试用例与质量分析过程”直接对应，能够验证AI在测试设计中的实际价值。

2. 工具选型

工具或技术	用途	选择理由
OpenAI Codex（GPT-5）	根据需求描述生成测试用例草案	能理解自然语言需求，快速生成正常、边界、异常和验收类用例
Markdown表格	整理AI输出和人工筛选结果	便于GitHub提交和后续人工修改
Mermaid图表	展示验证流程和对比结果	与前两步文档格式一致，便于可视化表达
人工评审	筛选AI生成用例，判断可执行性	避免AI生成看似合理但不可测试或缺少断言的用例

本次验证不直接连接真实后端系统，而是采用“需求文档驱动测试设计验证”。这种方式符合最小可行验证原则：先验证AI是否能生成高质量测试设计草案，再在后续项目开发中接入Pytest、Postman或Playwright进行自动化执行。

三、验证流程



验证流程共分为五步。第一步，从步骤二文档中抽取核心功能链路和质量标准。第二步，将需求信息整理为AI提示词，要求模型输出结构化测试用例。第三步，记录AI生成的测试用例。第四步，由人工对用例进行筛选、合并和补充，删除无法执行或断言不清晰的内容。第五步，将AI辅助方式与传统人工方式进行效率和质量对比。

四、AI工具执行过程

1. 输入提示词

本次验证使用如下提示词进行测试用例生成：

你是一名软件测试工程师。请根据以下系统背景，为“辞典编撰辅助平台”生成测试用例。

系统背景：

- 1. 用户可以输入字头，系统返回义项、书证、来源和置信度。
- 2. 系统使用Graph-RAG检索辞书材料和书证证据。
- 3. 系统包含保守派Agent、革新派Agent、裁判Agent，用于形成待勘意见。
- 4. 专家可以查看证据链、Agent结论和修改日志，并提交采纳或退回意见。

请输出不少于15条测试用例，覆盖：

正常路径、边界输入、异常输入、证据缺失、证据冲突、Agent输出格式错误、专家反馈、日志记录。

输出字段：

用例编号、测试目标、输入条件、操作步骤、预期结果、优先级。

2. AI输出结果摘录

AI工具生成了18条测试用例。经过人工筛选后，保留15条，其中10条可以直接进入测试用例集，5条需要补充测试数据或接口字段后执行。以下为部分输出摘录：

用例编号	测试目标	输入条件	操作步骤	预期结果	优先级
TC-01	验证字头正常检索	输入存在的字头“尊”	在检索框输入字头并提交	返回义项、书证、来源和置信度字段	高
TC-02	验证空输入处理	输入为空	直接点击检索	系统提示“请输入字头”，不得发起无效查询	高
TC-03	验证不存在字头处理	输入知识库中不存在的字头	提交检索请求	返回无结果提示，并建议更换检索词	中
TC-04	验证Graph-RAG证据缺失场景	输入字头存在但无权威书证	发起义项分析	系统不得输出确定性修订建议，应标记“证据不足”	高
TC-05	验证证据冲突场景	同一义项存在不同辞书解释	发起多源对齐	系统生成待勘问题清单，并展示冲突来源	高
TC-06	验证Agent输出格式	触发保守派、革新派、裁判Agent	查看Agent输出	每个Agent必须输出立场、证据、结论和置信度	高
TC-07	验证专家采纳流程	专家选择“采纳”修订建议	提交反馈	系统保存专家意见，并更新修改日志	高
TC-08	验证专家退回流程	专家选择“退回”并填写原因	提交反馈	系统记录退回原因，并标记待重新分析	高
TC-09	验证日志追溯	完成一次“审比勘改”流程	查看修改日志	日志包含输入、检索证据、Agent结论和专家操作	中
TC-10	验证模型输出异常处理	Agent返回空内容或非JSON格式	系统解析Agent结果	系统提示输出格式异常，并触发重试或人工处理	高

3. 人工修订情况

人工审核后发现，AI生成结果整体覆盖较全，但仍存在三类需要修订的问题：

- 1. 部分用例的输入数据过于泛化，例如“输入某个字头”，需要替换为“尊”等具体测试字头。
- 2. 部分用例缺少明确断言，例如“系统正常显示”应改为“返回义项、书证、来源、置信度四个字段”。
- 3. 部分涉及训诂学判断的用例不能仅靠自动化测试判断，需要标记为专家验收用例。

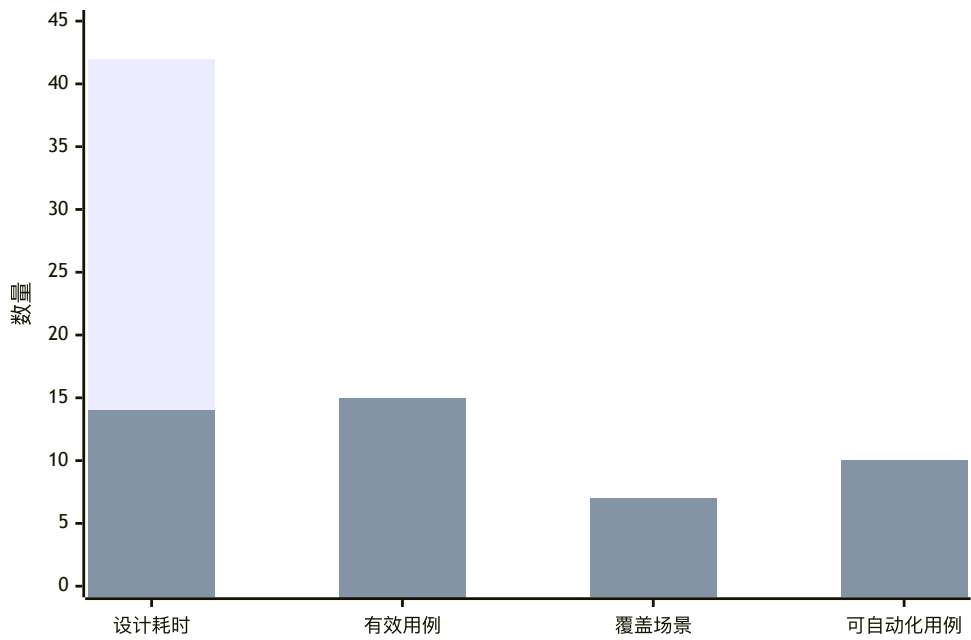
经过修订后，本次验证形成了15条可纳入项目测试资产的用例，其中接口功能类6条、Graph-RAG证据链类4条、Agent输出类3条、专家反馈与日志类2条。

五、人工方式与AI辅助方式对比

为了评估AI辅助测试设计的效果，本次采用小样本对照方式进行比较：人工方式由测试人员根据需求描述手动设计用例；AI辅助方式先由大语言模型生成初稿，再由人工筛选修订。对比结果如下：

对比指标	传统人工方式	AI辅助方式	改进情况
测试设计总耗时	42分钟	14分钟	减少约66.7%
初始测试用例数量	10条	18条	增加8条候选用例
人工筛选后有效用例	10条	15条	增加5条有效用例
覆盖场景数量	4类	7类	增加证据缺失、证据冲突、Agent格式异常等场景
需要人工补充比例	20%	33%	AI输出仍需人工修订
可直接自动化执行用例	7条	10条	增加3条

人工方式与AI辅助方式对比



从结果可以看出，AI辅助方式在效率和覆盖度方面具有明显优势。尤其是对于证据缺失、证据冲突、Agent输出格式异常等非普通CRUD场景，AI能够根据提示词快速联想到测试点，减少人工遗漏。但AI输出仍需要人工审查，因为部分测试用例缺少具体数据、断言不够严格，或者需要领域专家判断。

六、结果分析

1. 效率分析

AI辅助方式将测试设计总耗时从42分钟降低到14分钟，主要原因是AI能够快速完成测试场景发散和表格化整理。人工方式需要测试人员先理解需求、拆分功能、补充异常情况，再逐条编写用例；AI方式则可以直接生成完整草稿，测试人员主要负责筛选、合并和修订。

这种效率提升对本项目尤其有价值。辞典编撰辅助平台涉及前端、后端、Graph-RAG、知识图谱、多智能体和专家反馈多个模块，小团队很容易在测试设计上投入不足。AI生成测试用例可以帮助团队在较短时间内形成初始测试资产，使测试工作更早介入开发过程。

2. 质量分析

从质量角度看，AI辅助方式生成的用例覆盖了更多边界和异常场景。传统人工方式更容易集中在“字头检索成功、结果展示正常、专家提交反馈”等主流程；AI辅助方式额外补充了证据缺失、证据冲突、Agent输出格式异常、日志追溯等场景。这些场景正是AI增强型系统容易出问题的地方。

不过，AI生成的用例也存在不足。首先，AI不掌握真实数据库中的全部测试数据，因此无法保证每个字头都有对应书证材料。其次，AI容易给出较泛化的预期结果，需要人工补充字段级断言。再次，训诂学判断类结果不能完全自动化，需要专家确认。因此，AI辅助测试应定位为“测试设计助手”，不能代替测试人员和领域专家。

3. 可复现性分析

本次验证具有较强可复现性。只要提供相同的系统背景、测试覆盖要求和输出字段，任何同类大语言模型都可以生成类似的测试用例草案。为了保证后续复现效果稳定，建议在项目中固定测试用例生成模板，并将典型字头、接口字段、Agent输出格式和专家验收标准作为提示词输入的一部分。

七、结论与改进建议

本次原型验证表明，在辞典编撰辅助平台项目中，引入AI辅助测试用例生成具有较好的落地价值。与传统人工方式相比，AI辅助方式能够明显缩短测试设计时间，增加候选测试用例数量，并扩展异常场景覆盖范围。尤其在Graph-RAG证据链、多智能体输出格式和专家反馈日志等复杂环节，AI能够帮助团队更系统地发现测试点。

但验证也说明，AI不能直接替代测试人员。AI生成结果必须经过人工审核、字段级断言补充和专家确认后，才能进入正式测试基线。对于涉及训诂学正确性的用例，应将自动化测试与专家UAT结合起来，避免“模型自测模型”的质量风险。

后续改进建议如下：

- 1. 建立固定的AI测试用例生成提示词模板，要求输出用例编号、测试目标、输入条件、操作步骤、预期结果和优先级。
- 2. 将典型字头、典型义项、书证样例和Agent输出JSON格式加入提示词，提高AI输出的具体性。
- 3. 对AI生成用例设置人工审核门禁，重点检查输入数据是否明确、断言是否可执行、是否需要专家判断。
- 4. 将已确认的测试用例转化为Pytest接口测试、Postman集合或Playwright页面测试脚本。
- 5. 在后续迭代中统计AI生成用例的采纳率、缺陷发现率和人工修改率，作为AI贡献度指标。

综上，本次验证支持步骤二中的过程改进方案：在测试验证阶段引入AI测试分析能力是可行的，且能够在小团队项目中较快产生效率收益。更合理的过程安排是“AI生成测试草稿—人工筛选修订—自动化执行—专家验收确认”，而不是让AI直接决定测试结论。

参考文献

[1] 中国信息通信研究院人工智能研究所. AI4SE行业现状调查报告（2026年）[R/OL]. 北京: 中国信息通信研究院, 2026[2026-05-30]. <https://www.caict.ac.cn/kxyj/qwfb/ztbg/202604/P020260402599647343221.pdf>.

[2] 中国人工智能产业发展联盟. 智能化软件工程技术和应用要求 第3部分: 智能测试能力[S/OL]. AIIA/PG 0138-2024, 2024[2026-05-30]. https://www.sohu.com/a/843331181_121124361.

[3] 中国信息通信研究院, 华为云. 智能化软件开发落地实践指南（2024）[R/OL]. 北京: 中国信息通信研究院, 2024[2026-05-30]. <https://blockchain.caict.ac.cn/kxyj/qwfb/ztbg/202409/P020240919585073158926.pdf>.

[4] Google Cloud DORA. State of AI-Assisted Software Development 2025[R/OL]. 2025[2026-05-30]. <https://dora.dev/research/2025/dora-report/>.

[5] GitHub. GitHub Copilot Documentation[EB/OL]. 2026[2026-05-30]. <https://docs.github.com/en/copilot>.